Les Instructions Itératives (Les Boucles)

- Les Instructions Itératives
- La syntaxe et la sémantique de Pour
- La syntaxe et la sémantique de Tant-que
- La syntaxe et la sémantique de Répéter



James ADEOLA

Collège de Paris Supérieur : Département IT

Année Universitaire 2024-2025

Boucles - Introduction

Introduction

- Dans certaine situation, on est amener à répéter l'exécution d'une ou plusieurs instructions.
- Soit l'exemple suivant :

```
Algorithme Exemple1

Début
        Ecrire(1)
        Ecrire(2)
        Ecrire(3)
        Ecrire(4)
        Ecrire(5)
Fin
```




```
Program Exemple1;
Uses wincrt;

Begin
    Write(1)
    Write(2)
    Write(3)
    Write(4)
    Write(5)
End.
```

2) Ecrire un algorithme / programme qui affiche les nombre de 1 à 1000 ?

Boucles – Pour (1/4)

Syntaxe de la boucle Pour

- L'instruction de boucle Pour permet de répéter l'exécution d'un bloc d'instructions. Elle utilise un compteur d'itération, une valeur initiale et une valeur finale du compteur. Le compteur est incrémenté automatiquement
- La syntaxe de l'instruction de la boucle Pour, est :

Pour <cpt> ← <vi> **à** <vf> **Faire**
<instruction_1>
<instruction_2>
……..
<instruction_n>
Fin Pour

En PASCAL

For <Cpt> := <vi> **To** <vf> **Do**
Begin
<instruction_1>
<instruction_2>
……..
<instruction_n>
End;

Bloc du Pour

<cpt> : le compteur (Variable Entière). <vi> : Valeur initiale (valeur entière)
<vf> : Valeur finale (valeur entière)
Le nombre d'itérations (répétitions) = <vf> - <vi> + 1

Boucles – Pour (2/4)

Sémantique de la boucle Pour

- Soit l'exemple suivant (Algorithmique / Pascal)

Pour j←1 à 1000 **Faire**
    Ecrire (j)
Fin-Pour

for j:=1 **to** 1000 **do**
begin
    Writeln (j);
end;

- Ça veut dire quoi cette instruction ?
- Le sens de cette instruction est comme suit : L'instruction Ecrire(j) sera exécutée pour toutes les valeurs de j allant de 1 à 1000. Donc, elle sera exécutée 1000 fois. Le résultat de cette boucle est l'affichage des valeurs 1, 2, 3, …, 1000
- ***Remarque :***
  - Si le bloc d'instruction contient une seul instruction, on peut enlever les **begin** et le **end;** de l'instruction **for**.

Boucles – Pour (3/4)

Utilisation de la boucle Pour

- Quant-est-ce que nous utiliserons l'instruction de boucle Pour ?
- Dans toute formule mathématique qui utilise un ou plusieurs indices, comme les sommes, et dans les vecteurs, les matrices (types indicés)
  - Exemple : S = 1/2 + 2/3 + 3/4 + …. + n/(n+1)
  - T[1] ← 1*2 T[2] ← 2*3 T[3]←3*4 …. (voir vecteurs et matrices)
- En générale, si le nombre d'itérations (de répétitions) est connu (par exemple Nb), on peut utiliser la boucle Pour comme suit : **Pour** i←1 **à** Nb **faire** …



Boucles – Pour (4/4)

Exemple

- Ecrire un algorithme/Programme qui affiche tous les nombres entre deux nombres entiers N1 et N2 et qui sont divisibles par un nombre entier d.

```
Algorithme exemple_1
Variables
        N1, N2, d, i : entier
Début
   Lire (N1, N2, d)

   Pour i←N1 à N2 faire
      Si (i mod d = 0) alors
            Ecrire (i);
      Fin-Si
   Fin-Pour
Fin
```

```
Program exemple_1;
Uses wincrt;
Var
        N1, N2, d, i : integer;
Begin
   Read (N1, N2, d);

   for i := N1 to N2 do
     if   (i mod 2 = 0) then
            Writeln(i);
End.
```

Pas de point-virgule après **do**

Boucles – Tant-que (1/4)

Syntaxe de la boucle Tant-que

- L'instruction de boucle Tant-que permet de répéter l'exécution d'un bloc d'instructions. À chaque itération, une expression booléenne (condition) est réévaluer :
  - Si l'expression donne TRUE : donc on exécute l'itération et ensuite en réévalue l'expression booléenne
  - Si l'expression donne FALSE : on arrête la boucle Tant-que
- La syntaxe de l'instruction du boucle Tant-que est :

Tantque <condition> **Faire**
<instruction_1>
<instruction_2>
……..
<instruction_n>
Fin Tant-que

En PASCAL

While <Condition> **Do**
Begin
<instruction_1>
<instruction_2>
……..
<instruction_n>
End;

Bloc du Tantque

Boucles – Tant-que (2/4)

Sémantique de la boucle Tant-que

- Soit l'exemple suivant (Algorithmique / Pascal)
  - j ← 1

Tant-que j <= 1000 **faire**
    Ecrire (j)
    j ← j + 1
Fin-Tantque

while j <= 1000 **do**
begin
    Writeln (j);
    j := j +1;
end;

- Ça veut dire quoi cette instruction ?
- Le sens de cette instruction est : Tant-que la valeur de j est inférieure ou égale à 1000 les deux instructions Ecrire(j) et j←j+1 sont exécutées.
- La valeur initiale de J est 1, donc les valeurs 1, 2, …, 1000 seront affichées

Boucles – Tant-que (3/4)

Utilisation de la boucle Tant-que

- Quant-est-ce que nous utiliserons l'instruction de boucle Tant-que ?
- Dans tous les cas où la boucle **Pour** est appliquée, nous pouvons utiliser la boucle **Tant-que**.
- Si la boucle dépend d'une condition compliquée (expression booléenne avec opérateur logiques par exemple : **r<>0 et d<=n**)
- Il faut s'assurer de l'initialisation de la condition. (par exemple **r<>0 et d<=n** la variable r doit être initialisée avec une valeur non nulle) et s'assurer qu'il y a une instruction qui rend la condition fausse après un certain nombre d'itérations.



Boucles – Tant-que (4/4)

Exemple

- Ecrire un algorithme/Programme qui affiche tous les nombres entre deux nombres entiers N1 et N2 et qui sont divisibles par un nombre eniter d.

```
Algorithme exemple_2
Variables
        N1, N2, d, i : entier
Début
   Lire (N1, N2, d)
   i ← N1
   Tant-que i <= N2 faire
      Si (i mod d = 0) alros
            Ecrire (i);
      Fin-Si
      i ← i + 1
   Fin-Pour
Fin
```

```
Program exemple_2;
Uses wincrt;
Var
        N1, N2, d, i : integer;
Begin
   Read (N1, N2, d);
    i := N1;
    While i <= N2 do
     Begin
        if   (i mod d = 0) then
             Writeln(i);
        i := i+1
     End;
End.
```

Pas de point-virgule après **do** : ça peux causer une boucle infinie

Boucles – Répéter (1/4)

Syntaxe de la boucle Répéter

- L'instruction de boucle Répéter permet de répéter l'exécution d'un bloc d'instructions. À chaque itération, une expression booléenne (condition) est réévaluer :
  - Si l'expression donne TRUE : donc on arrête la boucle et on exécuter l'instruction qui vient après ***Répéter***;
  - Si l'expression donne FALSE : on continue la boucle en exécutant l'itération suivante
- La syntaxe de l'instruction du boucle Tant-que est :

Répéter
<instruction_1>
<instruction_2>
……..
<instruction_n>
Jusqu'à <condition>

En PASCAL

Repeat
<instruction_1>
<instruction_2>
……..
<instruction_n>
Until <condition>;

Bloc de Répéter

Boucles – Répéter (2/4)

Sémantique de la boucle Répéter

- Soit l'exemple suivant (Algorithmique / Pascal)
  - j ← 1

Répéter
Ecrire (j)
j ← j + 1
Jusqu'à j > 1000

Repeat
Writeln (j);
j := j +1;
Jusqu'à j > 1000;

- Ça veut dire quoi cette instruction ?
- Le sens de cette instruction est : On répète l'exécution des deux instructions Ecrire(j) et j←j+1 jusqu'à avoir la valeur de j strictement supérieur à 1000.
- La valeur initiale de J est 1, donc les valeurs 1, 2, …, 1000 seront affichées.

Boucles – Répéter (3/4)

Utilisation de la boucle Répéter

- Quant-est-ce que nous utiliserons l'instruction de boucle Répéter ?
- Dans tous les cas où la boucle **Pour** est appliquée, nous pouvons utiliser la boucle **Tant-que**.
- Si la boucle dépend d'une condition compliquée (expression booléenne avec opérateur logiques par exemple : **r<>0 et d<=n**)
- En général, si la première itération est réalisée sans condition, on peut utiliser l'instruction **Répéter** au lieu de l'instruction **Tant-que**.
- L'instruction **Répéter** est équivalente à **Tant-que**.



Boucles – Répéter (4/4)

Exemple

- Ecrire un algorithme/Programme qui affiche tous les nombres entre deux nombres entiers N1 et N2 et qui sont divisibles par un nombre eniter d.

```
Algorithme exemple_2
Variables
        N1, N2, d, i : entier
Début
   Lire (N1, N2, d)
   i ← N1
   Répéter
      Si (i mod d = 0) alros
            Ecrire (i);
      Fin-Si
      i ← i + 1
   Jusqu'à  i>N2
Fin
```

```
Program exemple_2;
Uses wincrt;
Var
        N1, N2, d, i : integer;
Begin
   Read (N1, N2, d);
    i := N1;
    Répéter
       if   (i mod d = 0) then
            Writeln(i);
        i := i+1
    Until  i>N2;
End.
```

Résumé

- Les instructions itératives (de boucle) permettent de répéter l'exécution d'un bloc d'instructions (une ou plusieurs instructions) un nombre d'itérations (un nombre de fois). Il y a trois instructions de boucle : **Pour** (**For**), **Tant-que** (**While**) et **Répéter** (**Repeat**).
- L'instruction de boucle **Pour** utilise un compteur (variable entière), une valeur initiale (valeur entière) et une valeur finale (valeur entière). Le nombre d'itérations est :
  - **VF - VI + 1** *si* VF >= VI
  - **0** *si* VF < VI (Aucune Itération)
- L'instruction de boucle **Tant-que** utilise une condition de boucle : si la condition est vérifiée on exécute l'itération, sinon on quitte la boucle. À la fin de chaque itération la condition est réévaluée.
- L'instruction de boucle **Répéter** utilise une condition d'arrêt de boucle : si la condition est vérifiée on exécute pas l'itération et on quitte la boucle, sinon on exécute l'itération suivante.